SEO Automation

What this tool is, what is finished, and what is left

Project folder: D:\Acentecom\seo-automation · Written 13 August 2026 · Every number below was read from the live database and the real code on that date.

> **In one sentence:** this tool crawls your website, pulls in your Google Search Console data, finds what is wrong with your SEO, and writes the fixes for you — so you can see every page, every problem, and every fix in one dashboard instead of clicking through Google's website.

12,807	81,486	93,354	42,851	16
Pages crawled	Traffic records	SEO issues found	Fixes written	Database tables

1. What the project is meant to do

The goal is a single dashboard that answers the questions an SEO owner asks every day, without logging into Google Search Console and clicking through ten screens:

- **What pages do I have?** — crawl the whole site and record every page.
- **Which ones does Google actually show?** — pull real traffic data from Search Console.
- **Which ones are broken?** — missing titles, duplicate content, no description, slow pages.
- **What should I change?** — write the actual replacement title or description, ready to approve.
- **Why is a page not on Google?** — show the reason, in plain words.

It is built for one owner running their own sites. Right now it is tracking **11 websites**, with the main one being `www.atmhtml5games.com`.

2. The technology used, and why

Nothing exotic. Every piece was chosen because it is standard, well documented, and boring in a good way.

The main building blocks

Tool	Version	What it does here, in plain words
`TypeScript`	6.0	The language everything is written in. It catches mistakes before the code ever runs.
`React`	19.2	Builds the screens you click on.
`Vite`	8.2	Bundles the screens into files a browser can load. Also runs the local preview.
`Express`	5.2	The server. Everything the screens ask for goes through it.
`PostgreSQL` + `pg`	8.23	The database where every page, number and fix is stored.
`Drizzle ORM`	0.45	Lets the code talk to the database safely, and keeps a written history of every table change.
`Crawlee`	3.18	The crawler engine. Walks the site link by link.
`Playwright`	1.62	A real browser, used for pages that only build themselves with JavaScript.
`Anthropic SDK`	0.116	Connects to Claude, which writes the improved titles and descriptions.
`Zod`	4.4	Checks that incoming data is the right shape before it is trusted.
`jsonwebtoken`	9.0	Keeps you logged in.

Outside services it talks to

Service	How it logs in	What we get from it
Search Console — Search Analytics	Google login (OAuth)	Clicks, impressions, position, per page and per search term
Search Console — Sitemaps	Google login (OAuth)	Which sitemaps Google has, and any errors in them
Search Console — URL Inspection	Google login (OAuth)	Whether one page is indexed, and the reason if not. Limit: 2,000 pages per day.
PageSpeed Insights	API key	Real page speed scores from actual Chrome users
Google Safe Browsing	API key	Whether Google has flagged the site for malware or scams
Claude (Anthropic)	API key	Writes better titles, descriptions, headings and image alt text

> **Important thing to understand about Google.** Google does *not* offer an API for everything you see on the Search Console website. There is no way to download the "Why pages aren't indexed" table in bulk, no Links API, no Mobile Usability API (Google switched it off in December 2023), and no Manual Actions API. Wherever Google refuses to give the data, this tool works it out from our own crawl instead — and every screen says clearly where its numbers came from, so nothing is ever passed off as Google's word when it isn't.

3. How the system is arranged

There are four separate parts. They run independently, which is why a long crawl never freezes the dashboard.

Part	Started with	Its job
The screens	`npm run dev:client`	What you look at and click. Runs in the browser on port 5173.
The server	`npm run dev:server`	Answers the screens, talks to Google, reads and writes the database.
The worker	`npm run worker`	Does the slow work — crawling sites — on its own, so nothing else has to wait.
The database	PostgreSQL	Holds everything, permanently.

How a crawl actually runs

You press *Crawl*. The server writes a job into the database and answers you immediately. The worker notices the job and starts. It reads `robots.txt` and the sitemap to find out where it is allowed to go, then walks the site link by link, saving every page. If a page looks empty because it builds itself with JavaScript, the worker opens it again in a real browser and saves the finished version. When it is done, the rules engine reads all the pages and records what is wrong. Then, if you ask, Claude writes the fixes. The dashboard shows progress the whole time.

How your Google login is protected

Google gives us a long-lived token so we can fetch your data without you logging in every time. That token is **encrypted before it is written to the database**, so even someone reading the database directly cannot use it.

4. What is in the database

Sixteen tables. These are the real counts as of 13 August 2026.

Table	Rows	What is in it
`users`	1	Your login
`websites`	11	The sites being tracked
`crawls`	26	One row each time a crawl runs
`pages`	12,807	Every page found, with its title, description, word count, links and more
`issues`	93,354	Every SEO problem found
`optimizations`	42,851	Every suggested fix, waiting for your approval
`audit_events`	233	A history of important actions
`gsc_connections`	1	Your Google login, encrypted
`gsc_properties`	1	Which Search Console property is linked to which site
`gsc_page_metrics`	81,486	Daily clicks and impressions per page — 16 months of history
`gsc_breakdowns`	13,978	The same traffic split by search term, device and country
`gsc_url_inspections`	0	Google's index verdict per page — empty, see section 7
`gsc_inspection_attempts`	30	Counts how much of the daily 2,000 limit is used
`gsc_sitemaps`	11	Your sitemaps and their status — new today
`web_vitals`	25	Real page speed scores — new today
`security_checks`	1	Safe Browsing result — new today

5. What is finished and working

> **Verified on 13 August 2026:** the whole project passes its type check with zero errors (`tsc -b`), and the production build succeeds (`vite build` — 315 KB of code, 89 KB compressed). Both were run, not assumed.

Core features — all working

Feature	State	What it does
Login and accounts	Done	Password login, sessions, protected pages
Add and manage sites	Done	Add a site, detect its platform, delete it
Website crawler	Done	Reads robots.txt and sitemaps, walks the whole site, respects limits
JavaScript page rendering	Done	Re-opens JavaScript-built pages in a real browser so nothing is missed
SEO content extraction	Done	Pulls titles, descriptions, headings, images, links, structured data, word count
Duplicate detection	Done	Fingerprints each page's content and groups the matches
Issue detection	Done	93,354 issues found and grouped by type and severity
Rule-based fixes	Done	Fixes that need no AI — predictable and free
AI-written fixes	Done	Claude writes better titles, descriptions, headings, alt text
Approve or reject fixes	Done	Every fix waits for you — nothing is applied automatically
Live crawl progress	Done	Watch pages arrive while the crawl runs
Cancel a crawl	Done	Stop a crawl part-way and keep what it already found
Crash recovery	Done	If the worker dies, its abandoned crawl is picked up again automatically

Google Search Console — working

Feature	State	What it does
Connect Google account	Done	Log in with Google; the token is encrypted before storage
Traffic data	Done	81,486 daily records covering 16 months, from 18 Apr 2025 to 10 Aug 2026
Search terms, devices, countries	Done	3,378 search terms, 3 device types, 160 countries
All URLs view	Done	Crawl data and Google data side by side, sorted into five buckets
Pagination everywhere	Done	Six screens now page through *all* rows instead of stopping at a few hundred

Added today — the six new sections

Section	State	Where the data comes from	What you see
Sitemaps	Live	Google Search Console	All 11 sitemaps, 4,918 URLs submitted, zero errors and zero warnings
Core Web Vitals	Live	PageSpeed Insights	25 pages measured with real Chrome user data — loading speed, responsiveness, layout stability
Security	Live	Google Safe Browsing	Result: **clean, no threats found**. Plus direct links to Google's manual actions page.
Links	Live	Our own crawl	Most-linked pages, outside domains you link to, and orphan pages nothing links to
Enhancements	Live	Our own crawl	Structured data grouped by type, with sample pages
Mobile	Live	Our crawl + PageSpeed	How many pages set a mobile viewport, which ones don't, plus mobile speed scores
Coverage	Partly	Our crawl (+ Google later)	Seven of Google's ten "not indexed" reasons, calculated ourselves. See section 7.

6. What we found on your site

These came out of the work done today, checked directly against the database.

43 articles are hidden from Google by mistake. In total 5,010 pages carry a `noindex` tag. Of those, 4,966 are `/tag/` archive pages — that is correct and normal. But **43 are real `/post/` articles** marked `noindex` with no `follow`. If that was not deliberate, those 43 articles cannot appear in Google at all. Each one is a one-line fix.

3,814 pages are duplicates with no canonical tag. These pages share their content with another page and do not tell Google which version is the original, so Google has to guess. Google's own report currently lists only 140 in this category, which means it has not worked through the rest yet. This is the largest genuine SEO problem on the site.

Good news. All 11 sitemaps are error-free and warning-free. Safe Browsing reports the site clean. No broken pages, redirects, or server errors were found in the last crawl. Of 10,000 pages crawled, **4,912 are fully indexable**.

Page speed, measured on real visitors

Measurement	Your result	What it means
Largest Contentful Paint	3,687 ms	How long until the main content appears. Google wants under 2,500 ms — **this needs work**.
Interaction to Next Paint	211 ms	How fast the page reacts to a tap. Google wants under 200 ms — **just over the line**.
Cumulative Layout Shift	0.00	How much the page jumps while loading. **Perfect score.**
Overall verdict	Average	Not failing, but loading speed is the thing to fix first.

Note: most pages show the same speed numbers. That is Google giving a whole-site average, which it does when an individual page has too few visitors to measure on its own.

7. What is still pending

> **1. Google's index verdicts — blocked until tomorrow.** The `gsc_url_inspections` table is empty. This is my fault: earlier in this project I ran a delete command to clean up a test account, and it cascaded and destroyed 1,172 inspection records. They were not in any backup. The only way to rebuild them is to ask Google again, and Google allows 2,000 checks per property per day — today's allowance was already spent.
>
> **What to do:** after roughly 12:30 PM on 14 August, open the Indexing tab and press *Check All*. Google will re-check your pages and the verdicts will come back, along with the three missing Coverage reasons.

The three Coverage reasons still missing

Google's report has ten reasons. Seven are calculated from our own crawl and are working now. Three are judgements only Google can make, so they need the inspection run above:

- Crawled — currently not indexed
- Discovered — currently not indexed
- Duplicate, Google chose a different canonical than you

Until then those three rows show "*needs an inspection run*" rather than a zero, so they can never be mistaken for "no problem here".

Other work still open

Item	Priority	Detail
Click through the new tabs in a browser	Next	The code compiles and the data is real, but the seven new screens have not yet been opened and clicked by a person
Coverage tab on screen	Next	The data and the API are finished and tested; the visual tab is the last piece
Measure more pages for speed	Medium	Only 25 pages measured so far. Now that the API key is in place, much larger batches can run.
Automatic daily refresh	Medium	Today every refresh is a button press. A nightly schedule would keep it current on its own.
Export to CSV	Low	For sharing findings with a developer
Date filters on the new tabs	Low	To compare this month against last month
Automated tests	Low	Checks that run by themselves and catch breakages early

8. How the code is organised

47 server files and 14 screens. Grouped by what they do:

Folder	What lives there
`server/src/routes/`	The 42 web addresses the screens can call — login, sites, crawls, Search Console
`server/src/crawler/`	The crawler: walking the site, reading each page, re-rendering JavaScript pages
`server/src/gsc/`	Everything Google: login, traffic sync, inspection, sitemaps, speed, security, coverage
`server/src/analysis/`	The rules that decide what counts as an SEO problem
`server/src/optimization/`	Writing the fixes — both the simple rules and the Claude-written ones
`server/src/db/`	The database connection and the definition of all 16 tables
`server/src/lib/`	Shared helpers: web address cleanup, robots.txt, login, crawl limits
`src/components/`	The screens you see
`drizzle/`	16 numbered files recording every change ever made to the database structure

The Google modules, one line each

File	Job
`oauth.ts`	Google login and refreshing the token
`crypto.ts`	Encrypts the token before it is stored
`client.ts`	Talks to Google, with retries
`syncMetrics.ts`	Downloads clicks and impressions
`ensureRange.ts`	Fetches a date range only if it is missing
`inspectUrls.ts`	Asks Google if a page is indexed, and manages the 2,000/day limit
`mergedUrls.ts`	Joins crawl data to Google data into one table
`sitemapsSync.ts`	Downloads sitemap status new
`webVitals.ts`	Fetches real page speed scores new
`safeBrowsing.ts`	Checks for malware and scam warnings new
`siteInsights.ts`	Works out links, structured data and mobile readiness from our crawl new
`coverage.ts`	Works out why pages are not indexed new

9. Running it

Command	What it does
`npm run dev`	Starts the screens and the server together
`npm run worker`	Starts the crawler. Needed before any crawl will run.
`npm run build`	Builds the finished version for release
`npm run db:generate`	Writes a new database change file after editing the table definitions
`npm run db:migrate`	Applies those changes to the database
`npm run lint`	Checks code style

Settings needed in the .env file

Setting	Required	Used for
`DATABASE_URL`	Yes	Where the database is
`JWT_SECRET`	Yes	Keeps you logged in
`GOOGLE_CLIENT_ID` / `SECRET`	Yes	Google login
`GOOGLE_REDIRECT_URI`	Yes	Where Google sends you back after login
`ANTHROPIC_API_KEY`	Optional	AI-written fixes. Without it, rule-based fixes still work.
`PSI_API_KEY`	Optional	Page speed data
`GOOGLE_API_KEY`	Optional	Safe Browsing check

All seven are currently set and working.

10. Honest summary

Working and proven: crawling, issue detection, fix writing, Google traffic data, sitemaps, page speed, security, links, structured data, mobile readiness, and seven of the ten index-coverage reasons. The project compiles cleanly and builds for production. Every number in this document was read from the live database today.

Not yet proven: the seven new screens have not been opened in a browser and clicked through by a person. The code compiles and the data behind them is real and verified, but that final visual check is still outstanding.

Blocked: Google's own index verdicts, until the daily limit resets around 12:30 PM on 14 August 2026.

Data lost, and it will not come back: 1,172 URL inspection records were destroyed by a delete command I ran, and no backup contained them. They can only be rebuilt by asking Google again. Everything else — 81,486 traffic records, 13,978 breakdowns, 12,807 pages, 93,354 issues, 42,851 fixes and the Google connection — is present and was re-counted after today's database change to confirm nothing else was affected.

SEO Automation — project documentation, 13 August 2026.
Figures read directly from the live PostgreSQL database and the source code on that date. Type check and production build were run, not assumed.